



Delivery Versus Payment Audit Report

Version 1.0

Shadowy Creators

September 15, 2025

Delivery Versus Payment Audit Report

Shadowy Creators

September 15, 2025

Prepared by: Shadowy Creators Lead Auditors: - Mattia Papa - Alessandro Bergamaschi

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
 - Issues found
- Findings
 - High
 - Medium
 - Low
 - * [L-1] ERC-20 minimal validation let users exposed to malicious tokens
 - * [L-2] Token upgradeability allows post-creation malicious behavior
 - * [L-3] NFT transfers use ERC20 method instead of safe transfer
 - * [L-4] Unbounded loops enable gas-based denial of service
- Informational
 - [I-1] Off-chain stored XSS risk from settlement reference
- Gas

Protocol Summary

Delivery Versus Payment (DVP) is a permissionless protocol that enables atomic swaps of an arbitrary number of assets between multiple parties. The protocol supports native ETH, ERC-20 tokens, and ERC-721 NFTs in a single settlement. Users create settlements containing multiple flows (asset transfers), all involved parties must approve before execution, and transfers occur atomically - either all succeed or all fail. The protocol operates as a non-upgradeable singleton contract with no admin privileges, emphasizing decentralization and immutability.

Disclaimer

The Shadowy Creators team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The findings described in this document correspond to the following commit hash:

```
1 e377b72c9e72e408fdb3c3c78de6ecac5a368574
```

Scope

- `DeliveryVersusPaymentV1.sol`
- `DeliveryVersusPaymentV1HelperV1.sol`
- `IDeliveryVersusPaymentV1.sol`

Roles

There are no admin roles or privileged accounts in the DVP protocol. The system is entirely permissionless where any address can create settlements, only involved parties can approve their own participation, and anyone can execute fully-approved settlements.

Issues found

Severity	Number of issues found
High	0
Medium	0
Low	4
Info	1
Gas Optimizations	0
Total	5

Findings

High

None

Medium

None

Low

[L-1] ERC-20 minimal validation let users exposed to malicious tokens

Description: ERC-20 minimal validation is susceptible to false positives or false negatives, potentially exposing users to malicious tokens. 1. False Positives (Non-ERC20 contracts that pass validation) - Contracts that implement `decimals()` function but are not ERC20 tokens (e.g., custom contracts with decimal precision)

2. False Negatives (Legitimate ERC20 tokens that fail validation):

- Very old ERC20 tokens that don't implement the `decimals()` function (extremely rare)
- ERC20 tokens with non-standard `decimals()` implementations (e.g., returning different data types)
- Malicious tokens that intentionally break the `decimals()` function

Risks: - Malicious actors could deploy contracts that pass ERC20 validation but behave unexpectedly during execution - Status query failures: `getSettlementPartyStatus()` could revert if tokens don't implement `allowance()` or `balanceOf()` - Users should still verify token contracts independently before creating settlements

Recommended Mitigation: - Add calls in `_isERC20()` to other standard ERC20 functions

Acknowledgement: - The potential risk is acknowledged at lines 395-396 `DeliveryVersusPaymentV1.sol`

[L-2] Token upgradeability allows post-creation malicious behavior

Description: Upgradeable or governance-controlled tokens can become malicious after settlement creation, since the contract performs no re-validation of token contracts at execution time.

Risks: - Proxy-based tokens can be upgraded to malicious implementations that steal funds, revert transfers, or manipulate balances - Governance-controlled tokens can have their logic changed to block transfers or redirect funds - ERC20 tokens validated at creation via `decimals()` check can later remove this function or change behavior - Settlement parties may lose funds or have settlements become unexecutable due to post-creation token changes

Recommended Mitigation: - Consider re-validating token contracts at execution time - Add monitoring for token upgrade events - Document risks clearly for users dealing with upgradeable tokens

[L-3] NFT transfers use ERC20 method instead of safe transfer

Description: When transferring NFTs (`isNFT == true`), the contract casts to `IERC20` and uses `SafeERC20.safeTransferFrom()`, which resolves to basic `transferFrom()` for ERC721 tokens, bypassing the `onERC721Received` callback mechanism. Note that NFTs are validated via ERC165 `supportsInterface` check at creation time, ensuring only legitimate ERC721 contracts can be marked as NFTs.

Risks: - Missing `onERC721Received` callbacks for recipient contracts that expect to be notified of NFT receipts - Does not follow ERC721 best practices for safe transfers with callback verification - Primarily a UX/compliance issue rather than a security vulnerability, as legitimate ERC721s support basic `transferFrom`

Recommended Mitigation: - When `isNFT == true`, use `IERC721(flow.token).safeTransferFrom(flow.from, flow.to, flow.amountOrId)` instead of casting to `IERC20` - Import `IERC721` interface and add proper NFT handling logic

[L-4] Unbounded loops enable gas-based denial of service

Description: Multiple functions contain unbounded loops over flows array and settlement IDs arrays without gas limit considerations. Large settlements or batch operations can exceed block gas limits.

Risks: - Settlements with too many flows become unexecutable within block gas limits - Batch operations on many settlements can fail due to gas exhaustion - Griefing: malicious actors can create settlements with excessive flows to waste gas of executors - Legitimate large settlements may become permanently unexecutable

Recommended Mitigation: - Consider adding soft caps on number of flows per settlement - Document gas considerations clearly for users - Implement pagination for batch operations where feasible

Acknowledgement: - Known issue documented in README.md lines 121-122: "The current chain's block gas limit acts as a cap. In every case it is the caller's responsibility to ensure that the gas requirement can be met."

Informational**[I-1] Off-chain stored XSS risk from settlement reference**

Description: The `settlementReference` field accepts arbitrary strings that are stored on-chain and may be rendered in user interfaces without proper sanitization.

Risks: - Malicious actors can inject HTML/JavaScript into settlement references - If UIs render these strings without escaping, stored XSS attacks are possible - Particularly risky when attackers can create settlements involving other parties and bypass UI input validation

Recommended Mitigation: - UI implementations must sanitize/escape settlementReference before rendering - Consider Content Security Policy (CSP) headers - Optionally add on-chain length limits or character set restrictions

Gas

None